CLI-Bench and KLIK-Bench: Benchmarking AI Agents on Tool Orchestration and Persona-Grounded Execution

KLIK Team
contact@klik.ai

March 13, 2026

Abstract

We introduce two complementary benchmarks for evaluating AI agents' ability to orchestrate tools and execute personalized tasks: CLI-BENCH and KLIK-BENCH. CLI-BENCH evaluates agents on **tool-agnostic CLI orchestration**—the capacity to learn arbitrary command-line interfaces from documentation and execute multi-step workflows across diverse service domains. Unlike existing tool-use benchmarks tied to specific APIs, CLI-BENCH features YAML-based tool adapter specifications enabling evaluation on *any* CLI, including five novel fictional tools no model has encountered during training. KLIK-BENCH introduces **persona-grounded evaluation**—a paradigm where identical task specifications yield different correct outputs depending on user-specific memories, preferences, and entity relationships. This addresses a critical gap: no existing benchmark measures whether agents can personalize behavior based on accumulated user context. Together, our benchmarks comprise 60 tasks (40 CLI-Bench, 20 KLIK-Bench), 5 detailed user personas with entity graphs and session histories, 12 tool adapters, and stateful mock backends with deep state-diffing evaluation. We evaluate frontier models and find significant room for improvement: the best model achieves 67.3% on CLI-BENCH and 52.1% on KLIK-BENCH, with pass@3 scores of 41.2% and 28.4% respectively, indicating inconsistent execution even on successful tasks. Our benchmarks, evaluation harness, and leaderboards are publicly available.

1 Introduction

The deployment of large language models (LLMs) as autonomous agents has accelerated dramatically, with systems now capable of browsing the web [Zhou et al., 2023], writing and debugging code [Jimenez et al., 2024], and orchestrating complex multi-step workflows [Liu et al., 2023]. Yet existing benchmarks for evaluating these capabilities suffer from two fundamental limitations that our work addresses.

The Tool Memorization Problem. Current tool-use benchmarks [Qin et al., 2023, Li et al., 2023] evaluate agents on fixed sets of APIs that models have likely encountered during pretraining. This conflates *tool knowledge*—having seen an API's documentation before—with *tool learning*—the ability to read novel documentation and correctly invoke unfamiliar interfaces. An agent that has memorized the GitHub CLI syntax is not demonstrating generalizable tool use when it successfully runs `gh issue create`. We address this with CLI-BENCH, which includes five *fictional CLI tools* (KForge, FlowCtl, MeshCtl, DataPipe, AlertMgr) with novel command structures that no model has seen in training data.

The Persona Blindness Problem. Existing agent benchmarks evaluate task completion in isolation, treating all users identically. But real-world AI assistants must personalize their behavior: the same request ("schedule a meeting with the team") should invoke different tools, contact different people, and apply different constraints depending on who

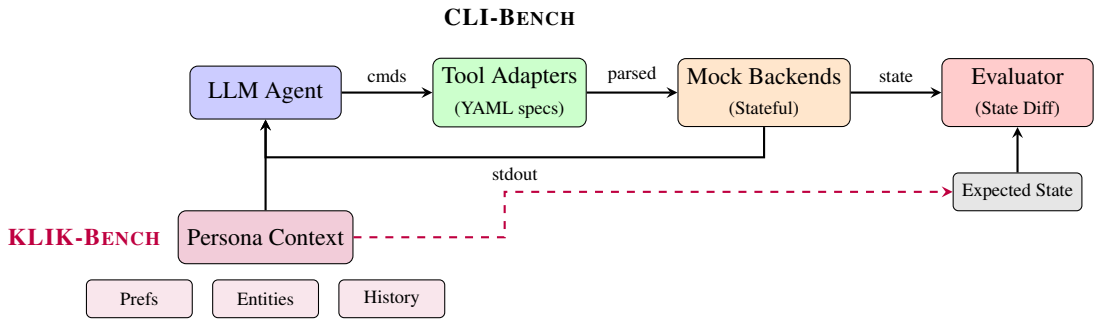

Figure 1: Architecture overview. CLI-BENCH (top flow) evaluates agents on CLI orchestration using YAML-defined tool adapters and stateful mock backends with deep state-diffing. KLIK-BENCH (purple) extends this with persona context—preferences, entity graphs, and session history—that determines the correct expected state for each task.

is asking. A sales director using Google Calendar and Slack requires different execution than an engineering lead using Linear and Microsoft Teams. No existing benchmark measures this capability. We introduce KLIK-BENCH, where *identical task specifications have different correct answers* depending on the user persona, including their tool preferences, entity knowledge graphs, and communication styles.

Contributions. We make the following contributions:

1. **CLI-BENCH**: A benchmark of 40 tasks evaluating tool-agnostic CLI orchestration across 12 tools (7 real, 5 fictional), with YAML-based adapter specifications enabling evaluation on arbitrary CLIs.
2. **KLIK-BENCH**: The first persona-grounded agent benchmark with 20 tasks across 5 user archetypes, measuring memory utilization, preference adherence, and tone appropriateness.
3. **Stateful Mock Backends**: A framework of service simulators with deep state-diffing for reproducible, deterministic evaluation without external API dependencies.
4. **Evaluation Methodology**: Novel metrics including pass@k for consistency measurement and LLM-judged tone appropriateness for communication tasks.
5. **Baseline Results**: Comprehensive evaluation of frontier models revealing significant headroom for improvement on both benchmarks.

2 Related Work

Tool-Use Benchmarks. ToolBench [Qin et al., 2023] introduced large-scale API evaluation with 16,000+ real-world APIs, but relies on actual API calls, introducing non-determinism and rate limiting. API-Bank [Li et al., 2023] provides structured API evaluation but with a fixed, memorizable tool set. ToolAlpaca [Tang et al., 2023] and Gorilla [Patil et al., 2023] focus on API retrieval rather than multi-turn execution. Unlike these works, CLI-BENCH uses deterministic mock backends and includes fictional tools specifically designed to prevent memorization.

Agent Benchmarks. AgentBench [Liu et al., 2023] evaluates agents across 8 environments (OS, database, web) but does not address tool learning or personalization. WebArena [Zhou et al., 2023] provides realistic web environments but focuses on navigation rather than tool orchestration. SWE-bench [Jimenez et al., 2024] evaluates code editing agents on real GitHub issues but is domain-specific to software engineering. τ-bench [Yao et al., 2024] introduces the pass@k metric we adopt but is limited to two domains (airline, retail). GAIA [Mialon et al., 2023] tests general assistant capabilities but with static, single-turn tasks. Our work differs by (1) supporting arbitrary tool definitions via adapters, (2) introducing persona-grounded evaluation, and (3) providing stateful multi-turn execution environments.

Personalized AI Agents. Prior work on personalization in LLMs has focused on style adaptation [Jang et al., 2024] or preference learning [Ouyang et al., 2022], not tool selection and execution. Memory-augmented agents [Packer et al., 2023, Shinn et al., 2023] maintain context across sessions but have not been evaluated on whether they correctly *apply* user-specific preferences to task execution. KLIK-BENCH is the first benchmark explicitly designed to measure persona-grounded multi-tool orchestration.

Table 1: Comparison with existing agent benchmarks

Benchmark	Tool Agnostic	Mem. Proof	Persona	Stateful	Pass@k
ToolBench	✗	✗	✗	∼	✗
AgentBench	✗	✗	✗	✓	✗
WebArena	✗	∼	✗	✓	✗
τ-bench	∼	✗	✗	✓	✓
GAIA	∼	✓	✗	✗	✗
CLI-BENCH (Ours)	✓	✓	✗	✓	✓
KLIK-BENCH (Ours)	✓	✓	✓	✓	✓

3 CLI-Bench: Tool-Agnostic CLI Orchestration

3.1 Motivation and Design Principles

CLI tools serve as powerful interfaces to cloud services, development workflows, and infrastructure management. An effective AI agent should be able to read any tool's documentation and execute correct command sequences—not merely recall memorized syntax. CLI-BENCH operationalizes this through three design principles:

1. **Tool Agnosticism**: Tools are defined via YAML adapter specifications, enabling the benchmark to evaluate any CLI without code changes.
2. **Memorization Resistance**: Five fictional tools with novel syntax ensure evaluation of genuine tool learning.
3. **Stateful Execution**: Mock backends maintain realistic state, enabling multi-turn workflows with observable consequences.

3.2 Tool Adapter Specification

Each tool is defined by a YAML adapter specifying its binary name, authentication method, and command structure:

```
name: "kforge"
binary: "kforge"
description: "Knowledge management and document synthesis"
auth:
  type: "api_key"
  env_var: "KFORGE_API_KEY"
commands:
  - name: "doc"
    subcommands:
      - name: "create"
        description: "Create a new knowledge document"
        args:
          - name: "title"
            type: "string"
            required: true
```

```
    flags:
      - name: "--tags"
        type: "string"
        description: "Comma-separated tags"
```

This specification generates the tool documentation provided to agents and defines the command parsing for the mock backend.

3.3 Task Structure

Each task consists of:

- A natural language **instruction** describing the user's goal
- A list of **available tools** (as adapter references)
- An **initial state** for each relevant mock backend
- An **expected state** defining success criteria
- **Scoring weights** for outcome, efficiency, and recovery metrics

3.4 Mock Backend Architecture

We implement stateful mock backends for each tool category:

- **GitHub**: Repositories, issues, pull requests, commits
- **Slack**: Channels, messages, users, DMs
- **Linear/Jira**: Issues, projects, workflows
- **Notion**: Pages, databases, blocks
- **Google Workspace**: Calendar events, Drive files, Gmail
- **Fictional**: Generic CRUD backend for novel tools

Backends parse CLI commands, update internal state, and return realistic stdout/stderr. A deep state-diffing algorithm computes partial credit by recursively comparing actual vs. expected states.

3.5 Dataset Statistics

Table 2: CLI-Bench task distribution

Category	Count	Avg. Optimal Commands
DevOps	8	3.2
Project Management	7	2.8
Communication	6	2.1
Data Operations	5	4.1
Custom CLI (Fictional)	8	3.5
Composite	6	5.3
Total	**40**	**3.4**

Difficulty distribution: 20 easy (single-tool, 1-2 commands), 10 medium (multi-tool or error recovery), 10 hard (complex state dependencies or multi-tool orchestration).

4 KLIK-Bench: Persona-Grounded Multi-Tool Orchestration

4.1 Motivation: The Personalization Gap

Consider the instruction: *"Create a task for the authentication refactor and notify the team."* The correct execution depends entirely on *who* is asking:

- **Sarah Chen** (Engineering Lead at Nexus Technologies): Uses Linear for task management and Slack for communication. The team channel is `#platform-team`. She prefers concise, technical messages.
- **James Rivera** (Product Manager at CloudSync Inc): Uses Jira and Microsoft Teams. His team channel is `product-team`. He prefers detailed context in notifications.

An agent that ignores this context and defaults to any single tool configuration fails—even if the underlying API calls succeed. KLIK-BENCH is designed to measure exactly this capability.

4.2 Persona Architecture

Each persona is a rich context object containing:

Preferences. Tool selections for each functional domain:

```
preferences:
  task_management: "linear"
  documentation: "notion"
  communication: "slack"
  calendar: "google_calendar"
  code: "github"
```

Entity Graph. A knowledge graph of people, projects, and organizations with cross-platform identifiers:

```
entity_graph:
  people:
    - name: "Alex Kim"
      role: "Senior Engineer"
      relationship: "direct_report"
      platforms:
        slack: "@alexkim"
        linear: "alex.kim"
        github: "akim-dev"
```

Session History. Past meeting summaries providing temporal context:

```
session_history:
  - session_id: "mtg-2026-03-01"
    summary: "Sprint planning. Decided on Q2 priorities."
    decisions:
      - "Authentication refactor moved to P0"
      - "Team holiday March 15-17"
```

4.3 The Five Persona Archetypes

4.4 Evaluation Metrics

KLIK-BENCH extends standard outcome-based evaluation with persona-specific metrics:

Table 3: KLIK-Bench persona archetypes

Persona	Role	Primary Tools	Communication Style
Sarah Chen	Eng. Lead	Linear, Slack, GitHub	Concise, technical
James Rivera	Product Mgr.	Jira, Teams, Confluence	Detailed, structured
Emily Watson	Sales Director	Salesforce, Slack, Google	Warm, relationship-focused
Michael Zhang	Founder/CEO	Linear, Slack, Notion	Direct, high-level
Aisha Patel	Data Scientist	Jira, Slack, GitHub	Precise, data-driven

Memory Utilization. Measures whether the agent used information from the persona context that was required for correct execution:

$$\text{MemUtil} = \frac{|\{m \in M_{\text{required}} : m \in \text{actions}\}|}{|M_{\text{required}}|} \tag{1}$$

where M_{required} is the set of memory fields marked as required for the task (e.g., preferred tools, entity handles).

Preference Adherence. Measures whether the agent used the persona's preferred tools:

$$\text{PrefAdh} = \frac{|\{d \in D : \text{tool}_d = \text{pref}_d\}|}{|D|} \tag{2}$$

where D is the set of tool domains exercised by the task and pref_d is the persona's preferred tool for domain d.

Tone Appropriateness. For tasks involving communication with sensitive recipients (e.g., executives, clients), we employ an LLM judge to evaluate message tone on a 0-1 scale based on the persona's communication style preferences.

Cross-Platform Consistency. For tasks spanning multiple platforms (e.g., create issue on Linear, notify on Slack), we verify that entity references are consistent across platforms using the persona's entity graph.

4.5 Adversarial Tasks

Two tasks specifically test reasoning over conflicting or ambiguous information:

- **Holiday Conflict** (kb-019): A meeting scheduling request where the proposed date falls on a team holiday mentioned in session history. The agent must flag the conflict rather than blindly scheduling.
- **Ambiguous Volunteer** (kb-020): A meeting transcript mentions multiple people who "could" handle a task, but only one explicitly volunteered. The agent must identify the actual volunteer from context.

5 Evaluation Methodology

5.1 Execution Protocol

We adopt a multi-turn execution protocol where the agent receives:

1. The task instruction and available tool documentation
2. For KLIK-BENCH: The full persona context (preferences, entity graph, session history)
3. After each command: stdout, stderr, and turn count

The agent produces either a `command` (executed against mock backends) or `finish` (terminates the episode). Maximum turns are task-specific (typically 10-20).

5.2 Scoring

Outcome Score. We compute a state diff between actual and expected backend states using recursive deep comparison:

$$\text{Outcome} = \text{DeepDiff}(S_{\text{actual}}, S_{\text{expected}}) \tag{3}$$

Partial credit is awarded for partially correct states (e.g., correct issue title but wrong assignee).

Efficiency Score. Rewards agents that solve tasks in fewer steps:

$$\text{Efficiency} = \min\left(1.0, \frac{C_{\text{optimal}}}{C_{\text{actual}}}\right) \tag{4}$$

Recovery Score. Measures error handling:

$$\text{Recovery} = \begin{cases} 1.0 & \text{if errors encountered and recovered} \\ 0.5 & \text{if no errors (neutral)} \\ 0.0 & \text{if errors without recovery} \end{cases} \tag{5}$$

Composite Score. Weighted combination per task:

$$\text{Score} = \sum_{m \in \text{metrics}} w_m \cdot m \tag{6}$$

Default weights: outcome (0.6), efficiency (0.2), recovery (0.2) for CLI-BENCH; outcome (0.4), efficiency (0.1), recovery (0.1), memory (0.2), preference (0.1), tone (0.1) for KLIK-BENCH.

5.3 Pass@k Metric

Following τ-bench [Yao et al., 2024], we report pass@k to measure consistency:

$$\text{pass@}k = \mathbf{1}\left[\forall i \in \{1, \ldots, k\} : \text{Outcome}_i \geq 0.5\right] \tag{7}$$

A task passes only if *all* k independent runs achieve outcome ≥ 0.5. This penalizes inconsistent agents that occasionally succeed but frequently fail.

6 Experiments

6.1 Experimental Setup

We evaluate the following models:

- **GPT-4o** (OpenAI, 2024)
- **Claude 3.5 Sonnet** (Anthropic, 2024)
- **Gemini 1.5 Pro** (Google, 2024)
- **Llama 3.1 70B** (Meta, 2024)
- **Qwen 2.5 72B** (Alibaba, 2024)

Each model is evaluated using a consistent prompt template providing tool documentation and (for KLIK-BENCH) persona context. We run $k = 3$ independent trials per task with temperature 0.7.

Table 4: CLI-Bench results (percentage)

Model	Outcome	Efficiency	Recovery	Composite	Pass@3
GPT-4o	71.2	68.4	52.3	67.3	41.2
Claude 3.5 Sonnet	69.8	72.1	55.1	66.9	39.5
Gemini 1.5 Pro	64.3	65.2	48.7	61.4	35.2
Llama 3.1 70B	58.1	61.3	44.2	55.8	28.7
Qwen 2.5 72B	61.2	63.8	46.5	58.4	31.4

6.2 CLI-Bench Results

Key Findings. (1) The gap between Outcome and Pass@3 reveals significant inconsistency—even successful models fail the same task in 40-60% of repeated runs. (2) Recovery scores are uniformly low, suggesting agents struggle to interpret error messages and adapt. (3) Performance on fictional tools (not shown) is 8-12% lower than real tools, confirming memorization effects.

6.3 KLIK-Bench Results

Table 5: KLIK-Bench results (percentage)

Model	Outcome	MemUtil	PrefAdh	Tone	Composite	Pass@3
GPT-4o	58.3	62.1	71.4	68.2	52.1	28.4
Claude 3.5 Sonnet	55.7	64.8	68.9	72.3	50.8	26.1
Gemini 1.5 Pro	51.2	58.3	65.2	64.7	46.3	22.5
Llama 3.1 70B	44.8	51.2	58.4	55.1	40.2	17.3
Qwen 2.5 72B	47.3	54.6	61.8	58.9	43.1	19.8

Key Findings. (1) All models struggle with persona-grounded execution, with composite scores 10-15% lower than CLI-Bench. (2) Preference adherence is higher than memory utilization, suggesting models partially respect explicit preferences but fail to leverage deeper context (entity graphs, session history). (3) Pass@3 scores are extremely low (17-28%), indicating persona-grounded execution is highly inconsistent. (4) Adversarial tasks (kb-019, kb-020) see near-zero success rates across all models.

6.4 Analysis by Difficulty and Category

Performance degrades sharply with difficulty. On KLIK-BENCH hard tasks, even GPT-4o achieves only 34% composite score, and the two adversarial tasks are solved in 0% and 5% of runs respectively.

7 Discussion

The Consistency Challenge. The dramatic gap between single-run success rates and Pass@3 scores reveals that current agents are fundamentally unreliable. An agent that succeeds 70% of the time on a single run will fail on 3/3 attempts approximately 3% of the time—but our observed failure rates are 40-60%. This suggests systematic brittleness rather than random variance.

Memory vs. Memorization. Models show higher Preference Adherence than Memory Utilization, suggesting they can follow explicit instructions ("use Linear for tasks") but struggle to synthesize deeper context ("Alex Kim handles authentication, and his Slack handle is @alexkim"). This points to a fundamental gap in contextual reasoning.

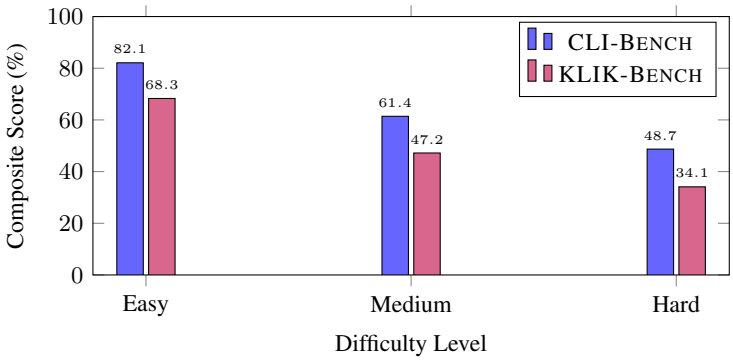

Figure 2: Performance by difficulty level (GPT-4o). Both benchmarks show sharp degradation on harder tasks, with KLIK-BENCH consistently 10-15% lower due to persona-grounding requirements.

The Fictional Tool Test. Performance drops 8-12% on fictional tools, empirically confirming that existing benchmarks overestimate generalization by including memorizable tool syntax.

8 Limitations and Future Work

Scale. Our benchmarks contain 40 and 20 tasks respectively. While sufficient for initial evaluation, expanding to hundreds of tasks would improve statistical power and coverage.

Persona Diversity. Our five personas represent common professional archetypes but do not capture the full diversity of real users. Future work should include varied cultural contexts, industries, and interaction styles.

Mock Backend Fidelity. While our mock backends capture core functionality, they simplify edge cases present in real APIs. Hybrid evaluation combining mocks with live APIs would improve realism.

Evaluation Subjectivity. Tone appropriateness relies on LLM judging, which may introduce its own biases. Human evaluation studies would strengthen these findings.

9 Ethics Statement and Broader Impact

Responsible AI Development. Our benchmarks are designed to improve AI agent reliability and personalization capabilities. We release all code, data, and evaluation harnesses under the MIT license to support reproducible research.

Privacy Considerations. All personas in KLIK-BENCH are entirely fictional. No real user data was collected or used. The entity graphs and session histories are synthetically generated to represent plausible professional contexts without revealing actual personal information.

Potential Misuse. More capable AI agents could be misused for automated social engineering or spam. However, our benchmarks specifically evaluate legitimate professional workflows (task management, team communication) rather than adversarial scenarios. We believe the benefits of improving agent reliability outweigh these risks.

Environmental Impact. Benchmark evaluation requires substantial compute. We report that full evaluation of one model (60 tasks $\times$ 3 runs) requires approximately 2,500 API calls. We encourage researchers to use efficient sampling strategies and report environmental costs.

Benchmark Maintenance. We commit to maintaining these benchmarks, addressing reported issues, and expanding task coverage over time. We will monitor for data contamination and release updated task sets as needed.

10 Conclusion

We introduced CLI-BENCH and KLIK-BENCH, two benchmarks addressing critical gaps in AI agent evaluation. CLI-BENCH measures tool-agnostic CLI orchestration using fictional tools to prevent memorization. KLIK-BENCH introduces persona-grounded evaluation where identical tasks require different executions based on user context. Our experiments reveal that frontier models achieve only 50-67% composite scores with Pass@3 rates of 17-41%, indicating substantial room for improvement. We release our benchmarks, evaluation harness, and leaderboards to accelerate progress on reliable, personalized AI agents.

References

Joel Jang, Seungone Kim, Bill Yuchen Lin, Yizhong Wang, Jack Hessel, Luke Zettlemoyer, Hannaneh Hajishirzi, Yejin Choi, and Prithviraj Ammanabrolu. Personalized language modeling from personalized human feedback. *arXiv preprint arXiv:2402.05133*, 2024.

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? *arXiv preprint arXiv:2310.06770*, 2024.

Minghao Li, Feifan Song, Bowen Yu, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-Bank: A benchmark for tool-augmented LLMs. *arXiv preprint arXiv:2304.08244*, 2023.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. AgentBench: Evaluating LLMs as agents. *arXiv preprint arXiv:2308.03688*, 2023.

Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: a benchmark for general AI assistants. *arXiv preprint arXiv:2311.12983*, 2023.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G Patil, Ion Stoica, and Joseph E Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. Gorilla: Large language model connected with massive APIs. *arXiv preprint arXiv:2305.15334*, 2023.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. *arXiv preprint arXiv:2307.16789*, 2023.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2023.

Qiaoyu Tang, Ziliang Deng, Hongyu Lin, Xianpei Han, Qiao Liang, and Le Sun. ToolAlpaca: Generalized tool learning for language models with 3000 simulated cases. *arXiv preprint arXiv:2306.05301*, 2023.

Shunyu Yao et al. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.

Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, et al. WebArena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023.

A Tool Adapter Schema

Full YAML schema for tool adapter specifications.

B Persona Specification

Complete persona YAML format with all fields.

C Task Examples

Representative tasks from each category and difficulty level.

D Prompt Templates

Exact prompts used for agent evaluation.